

Round-2 Hands-On Projects Submission Guide

This document outlines the recommended GitHub repository structure and the submission form design for the Round-2 Hands-On Projects: "Evaluation of Indic Translation Models" and "ASR-Based Transcription and Transliteration System."

1. GitHub Repository Structure

To ensure a professional, reproducible, and well-organized submission, the project should be structured as a research and engineering monorepo. The following directory and file structure is recommended:

Plain Text

```
indic-translation-asr-project/
|
├── README.md
├── requirements.txt
├── .gitignore
├── LICENSE
|
├── data/
│   ├── raw/
│   │   ├── translation_dataset.csv
│   │   └── sample_audio/
│   ├── processed/
│   │   ├── translated_outputs/
│   │   ├── token_analysis/
│   │   └── asr_outputs/
│   └── results/
│       ├── sacrebleu_scores.csv
│       ├── token_statistics.csv
│       └── plots/
|
├── docs/
│   ├── architecture/
│   │   ├── system_architecture.png
│   │   └── buffer_flow_diagram.png
│   ├── recordings/
│   │   └── demo_links.md
|
```

```
└─ reports/
  │─ project1_summary.pdf
  └─ project2_summary.pdf
└─ task1_translation_evaluation/
  │
  │─ README.md
  │
  │─ part_a_batch_translation/
  │ │─ part_a_translation_evaluation.ipynb
  │ │─ translation_outputs.csv
  │ │─ sacrebleu_results.csv
  │ │─ observations.md
  │
  │─ part_b_token_analysis/
  │ │─ part_b_token_eda.ipynb
  │ │─ token_counts.csv
  │ │─ engineered_features.csv
  │ │─ plots/
  │ │─ observations.md
  │
  └─ part_c_indic_token_behavior/
    │─ part_c_indic_token_analysis.ipynb
    │─ tokenization_comparison.csv
    │─ tamil_token_patterns.csv
    │─ plots/
    └─ observations.md
└─ task2_asr_transliteration/
  │
  │─ README.md
  │─ requirements.txt
  │─ Dockerfile
  │─ docker-compose.yml
  │
  │─ app/
  │ │─ main.py
  │ │─ asr_pipeline.py
  │ │─ transliteration.py
  │ │─ buffer_manager.py
  │ │─ interface.py
  │ │─ utils.py
  │
  │─ models/
  │ │─ model_config.py
  │
  │─ static/
  └─ templates/
```

```
├── sample_inputs/
│   └── sample.wav
├── outputs/
│   ├── transcripts/
│   └── transliterations/
├── tests/
│   └── test_pipeline.py
└── presentation/
    ├── final_presentation.pptx
    └── evaluation_video_link.md
```

Essential Repository Files

Ensure the following files are included at the root of your repository for clarity and reproducibility:

- `README.md` : A comprehensive overview of the project, including installation instructions, how to run each task, and a summary of results.
- `requirements.txt` : Lists all Python dependencies required for both projects.
- `.gitignore` : Specifies intentionally untracked files to ignore.
- `LICENSE` : Defines the licensing for your project.

2. Project-Specific Details and Recommendations

Project 1: Evaluation of Indic Translation Models

This project involves three parts, each requiring a dedicated Jupyter Notebook (`.ipynb`) for analysis and reporting. Key recommendations include:

Part A: Batch Translation and Evaluation

- **Model Selection:** Strongly recommended to use `ai4bharat/indictrans2-en-indic-1B` due to its optimization for Indic languages, superior Tamil linguistic handling, and Indic-aware tokenization. Other viable options include `facebook/nllb-200-distilled-600M` , `google/mt5-base` , and `Helsinki-NLP/opus-mt-en-ta` .
- **Dataset:** A provided dataset (e.g., `translation_dataset.csv` in `data/raw/`) should be used for batch translation into Tamil.

- **Evaluation:** Use the `sacreBLEU` score to evaluate translation quality. Report findings with accuracy metrics.
- **Notebook Flow:** The notebook (`part_a_translation_evaluation.ipynb`) should cover: Introduction, Model Choice Justification, Dependency Installation, Dataset Loading, Model Loading, Batch Translation, Output Saving, SacreBLEU Computation, Analysis, and Conclusion.

Part B: Token-Based Comparative Analysis

- **Model Selection:** Choose 5 diverse LLMs for comparison, such as `ai4bharat/indictrans2-en-indic-1B` , `facebook/nllb-200-distilled-600M` , `google/mt5-base` , `Helsinki-NLP/opus-mt-en-ta` , and `google/madlad400-3b-mt` .
- **Sentences:** Translate 5-10 sentences from English (or other languages) to Tamil using each model.
- **Metrics:** For each sentence and model, compute `source_token_count` , `target_token_count` , `expansion_ratio` (target/source), `avg_word_length` , `subword_fragmentation` , and `unknown_token_rate` .
- **EDA:** Include plots such as histograms of token counts, boxplots of expansion ratios, scatter plots of source vs. target tokens, and bar charts of average expansion per model.
- **Analysis:** Identify the model with the lowest token expansion rate and explain the underlying reasons (e.g., SentencePiece/BPE, Tamil morphology, subword fragmentation, Unicode handling).

Part C: Indic Token Behavior Analysis

- **Analysis Focus:** Investigate how Indic tokens (Tamil-specific linguistic units) are handled across the 5 selected models. Analyze subword splitting, average characters per token, Unicode fragmentation, rare token frequency, and memory footprint.
- **Insights:** Discuss the impact of Tamil's agglutinative structure, why English-centric tokenizers may fragment Tamil, the differences between SentencePiece and BPE, vocabulary coverage, and the effect of token explosion on Transformer memory.

Project 2: ASR-Based Transcription and Transliteration System

This project requires building a deployable system, containerized with Docker, and featuring an interactive interface.

Recommended Stack

- **ASR Model:** `openai/whisper-small` or `openai/whisper-medium` from Hugging Face for its multilingual capabilities and ease of integration.

- **Transliteration:** The `indic-transliteration` Python library for converting between scripts (e.g., English script to Tamil script, or Tamil to Latin transliteration).
- **Interactive Interface:** `Gradio` is highly recommended for its minimal code, Hugging Face friendliness, and audio upload support.

System Architecture

Plain Text

```
Audio Input
  ↓
Buffer Queue
  ↓
ASR Module
  ↓
Transcript
  ↓
Transliteration Engine
  ↓
User Interface
```

Buffer System

Implement a queue-based buffer (e.g., `queue.Queue()` in Python) to handle chunked audio, prevent overflow, and support asynchronous processing for real-time interaction.

Dockerization

Provide a `Dockerfile` and `docker-compose.yml` for reproducibility and modular deployment. The `Dockerfile` should include:

Plain Text

```
FROM python:3.10

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

CMD ["python", "app/main.py"]
```

task2_asr_transliteration/README.md Content

This README should include clear instructions for:

- **Installation:**

Bash

```
git clone <repo>
cd task2_asr_transliteration
pip install -r requirements.txt
```

- **Docker Build:**

Bash

```
docker build -t asr-system .
```

- **Docker Run:**

Bash

```
docker run -p 7860:7860 asr-system
```

- **Local Run:**

Bash

```
python app/main.py
```

3. Submission Form Design

The submission form should be compact and precise, requiring only two inputs:

1. **GitHub Repository Link:**

- **Input Type:** Text field
- **Example:** `https://github.com/yourusername/indic-translation-asr-project`
- **Explanation:** This link should point to the root of your public GitHub repository containing all project files and documentation as per the recommended structure.

2. **Links to Workings with Explanations (Google Drive):**

- **Input Type:** Text area (multi-line text input)
- **Example Content:**

Plain Text

Task 1 Demo Recording:

<https://drive.google.com/link-to-task1-demo>

Task 2 Demo Recording:

<https://drive.google.com/link-to-task2-demo>

Final Presentation (Optional):

<https://drive.google.com/link-to-presentation>

Short Description:

This repository contains:

1. Indic Translation Model Evaluation

- sacreBLEU evaluation
- token analysis
- Indic token behavior analysis

2. ASR + Transliteration System

- Whisper-based ASR
- Transliteration pipeline
- Dockerized deployment
- Interactive interface

- **Explanation:** This field should contain direct links to Google Drive recordings demonstrating the functionality and key findings of both projects. A brief description summarizing the contents of the repository and the key aspects of each project should also be included here. Ensure all shared drive links have appropriate access permissions.

4. Example Datasets and Information

While specific datasets are not provided in this guide, it is assumed that for Project 1, a batch translation dataset will be provided or sourced. For Project 2, sample audio files will be necessary for testing the ASR system. These should be placed in the `data/raw/` and `task2_asr_transliteration/sample_inputs/` directories respectively.

5. Conclusion

Adhering to this structured approach for both repository organization and submission will facilitate a clear and comprehensive evaluation of your projects. Good luck!